

Feature, Technology, Prompt & Policy Report

A complete, audit-ready walkthrough of the Seller Helper console — its seven tabs, the rule engine and LLM strategy behind it, the verbatim prompts, the quick-fix catalogue, and the policies enforced in code.

Generated 2026-05-12T21:01:57Z · Confidential draft

Module path: **/seller-helper**

Heuristic path: **src/server/seller-helper/***

Engine path: **src/server/conception/***

UI path: **src/components/SellerHelper/***

Abstract

This report documents the Seller Helper console exactly as it ships in the repository: the data foundation (pa_* micro-events), the seven dashboard tabs, the deterministic rule engine and its thresholds, the LLM strategy with provider fallback, the verbatim prompts, the per-product Vitrina heuristics and scoring formulas, the quick-fix catalogue, and the privacy and operational policies enforced by the code.

Table of contents

1. Executive summary
2. Product context and audience
3. Information architecture
4. Solution architecture and tech stack
5. Access control and routing policy
6. Data foundation: events, alerts, security blocks, applied actions
7. Dashboard (Overview) tab
8. Timeline tab
9. User Behavior tab
10. Conversion Funnel tab
11. Vitrina Recommendation tab
12. AI Recommendations tab
13. Alerts tab
14. Security tab
15. Rule engine — full catalogue
16. LLM strategy and provider fallback
17. Prompt library (verbatim)
18. Vitrina heuristics, scoring and ranking
19. Quick fixes (Vitrina and Security)
20. Refresh model and side effects
21. Privacy, integrity and operational policies
22. Appendices (source index, env, glossary)

Executive summary

Seller Helper is the operations console packaged inside Vitrina Store at the route `/seller-helper`. It gives privileged sellers a single page to read the storefront's pulse — traffic, funnel, behaviour, security — and to act on it with curated, schema-validated AI recommendations and reversible quick fixes.

The product combines a **deterministic rule engine** (five thresholds expressed directly in SQL/TypeScript) with a **large-language-model pass** (OpenRouter primary, Gemini fallback) that runs against a tight JSON snapshot of the latest signals. Every output is fingerprinted, idempotent, and scoped to fields that can be undone — there is no destructive automation.

- **Surface:** 7 tabs — Overview, Behavior, Funnel, Vitrina, AI Recommendations, Alerts, Security.
- **Engine:** rule engine on rolling 15-minute, 2-hour and 7-day windows; LLM analysis with retries and provider fallback.
- **Auditability:** alerts/recos carry a stable fingerprint per UTC day or hour; insert-on-conflict-do-nothing keeps re-runs safe.
- **Reversibility:** 4 Vitrina quick fixes and 2 security quick fixes — block/unblock + 4 catalogue edits, no event deletion.
- **Privacy posture:** session_key only, masked in UI, French-language operator copy on Security.

Product context and audience

Seller Helper targets store owners and operations admins of the Algerian e-commerce front end (Vitrina Store). The audience expects French operational copy on sensitive flows (Security) and DA-denominated prices, while keeping global analytics in English (en-US locale) to align with web analytics conventions.

Operationally the page has two visible jobs:

1. **Diagnose** — surface what is changing in traffic, conversion, behaviour and security right now.
2. **Act** — apply small, reversible mutations (Vitrina catalogue tweaks, session blocks) without leaving the page.

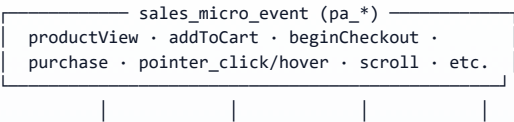
The hero contains a live data badge (*Live data · auto-refresh* when events are flowing) and a sessions-per-15-minute counter. Two CTAs live to the right: *Refresh* re-reads the precomputed overview; *Run analysis* triggers the rule engine + LLM pass server-side.

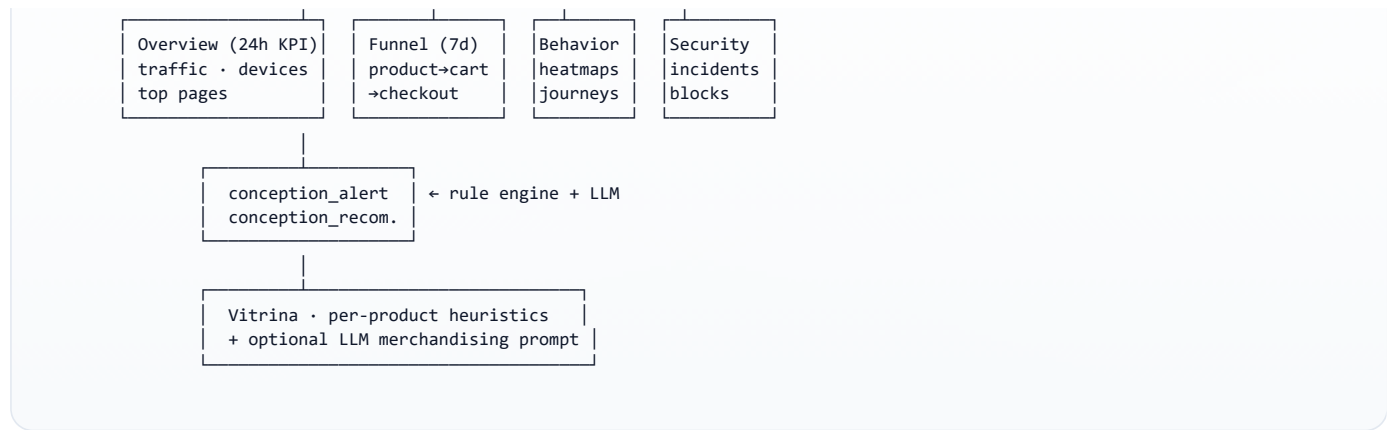
Information architecture

The dashboard exposes a deterministic 7-tab navigation. Tab metadata lives in `src/components/SellerHelper/nav.ts` — the source of truth for the labels and tooltip descriptions visible under the nav bar.

Tab	Label	Purpose	Primary source	Primary signal
Dashboard (Overview)	Overview	Traffic, devices, and top pages at a glance.	<code>src/components/SellerHelper/SellerHelperDashboard.tsx</code>	All <code>pa_*</code> micro-events in the last 24 h / 7 d window.
Timeline	Timeline	Plot any metric over time — whole store or a specific product.	<code>src/components/SellerHelper/timeline-tab.tsx</code> , <code>src/server/seller-helper/timeline-series.ts</code>	Bucketed <code>pa_*</code> micro-events over the selected range (24 h hourly · 7/30/90 d daily).
User Behavior	Behavior	Heatmaps, journeys, scroll depth, and session replays.	<code>src/components/SellerHelper/sections.tsx</code> , <code>ProductPageHeatmap.tsx</code>	<code>pa_scroll</code> , <code>pa_pointer_hover</code> , <code>pa_pointer_click</code> , <code>pa_product_view_time</code> .
Conversion Funnel	Funnel	Drop-offs from product view to payment.	<code>src/server/conception/metrics.ts</code> (funnelCounts)	<code>pa_product_view</code> → <code>pa_add_to_cart</code> → <code>pa_begin_checkout</code> → <code>pa_purchase</code> .
Vitrina Recommendation	Vitrina	Catalog and merchandising fixes by product.	<code>src/server/seller-helper/product-marketing-recommendations.ts</code>	Per-product aggregates over 7 days joined with the product catalogue.
AI Recommendations	AI	Prioritised actions derived from analytics signals.	<code>src/server/conception/llm-analysis.ts</code> , <code>analyze.ts</code>	Conception overview snapshot + funnel/security signals sent to the LLM.
Alerts	Alerts	Active incidents and recent resolutions.	<code>src/server/conception/analyze.ts</code> , <code>alert-detail-analysis.ts</code>	Rule engine on rolling windows of micro-events + LLM-emitted alerts.
Security	Security	Suspicious sessions, data integrity, and session blocking.	<code>src/server/conception/security-intel.ts</code> , <code>apply-security-quick-fixes.ts</code>	Per-session aggregates over 7 days, with 14-day reference window.

Figure 3 — Tabs as orthogonal lenses over the same DB



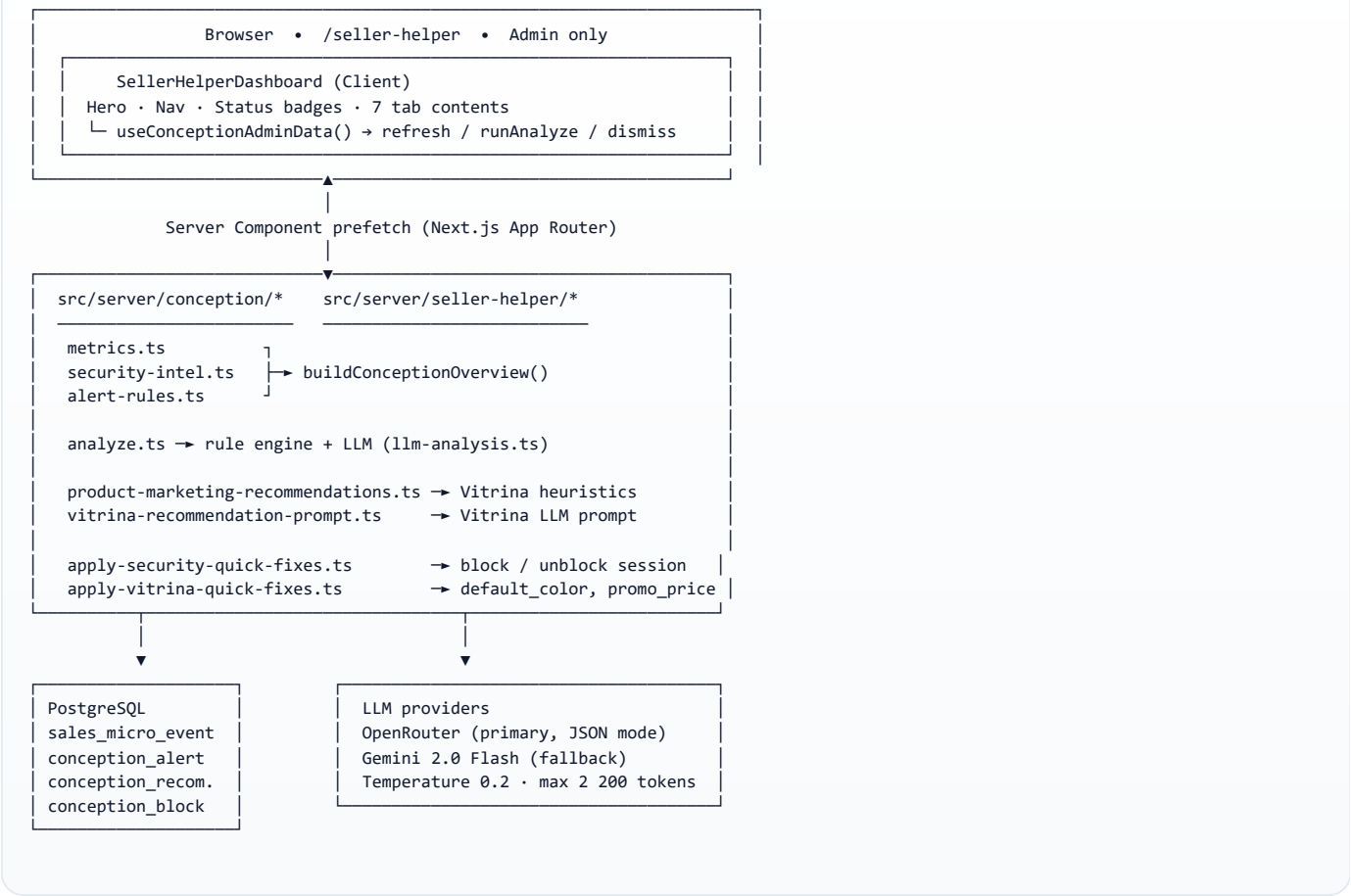


Each tab consumes data from the same overview DTO (`ConceptionOverviewDto`) plus its own slice. There is no tab-to-tab data fetching dependency: switching tabs is instantaneous because all data already lives in the client store after server hydration.

Solution architecture and tech stack

Seller Helper is rendered through Next.js App Router. The page is a server component that authenticates, joins the user role, prefetches all initial data in parallel, and passes it as `initialData` to a client dashboard.

Figure 1 — Seller Helper request and data flow



Stack at a glance

Layer	Technology / Library	Role inside Seller Helper
App Router page	Next.js 15 (React 19) — server component	Authenticates the request, gates admin access, prefetches data for hydration.
Client dashboard	React Client Component + Redux-aware hooks	Renders the 7 tabs and manages the active section via local state.
Styling	Tailwind CSS 3 + tailwind-merge + class-variance-authority	Cohesive panel/insight/badge primitives in layout.ts shared across tabs.
Icons	lucide-react	Section icons (Store, Compass, AlertTriangle, Shield, Bell, Zap, Users...).
Carousels	Swiper	Featured Vitrina recommendations slider with custom navigation.
Charts	Custom SVG (TrafficChart, ThreatActivityChart, ProgressBar)	Lightweight chart primitives in charts.tsx — no chart library dependency.

Layer	Technology / Library	Role inside Seller Helper
State sync	React state + custom useConceptionAdminData hook	Holds overview/alerts/recommendations/security data, exposes refresh/runAnalyze/dismiss handlers.
Server runtime	Next.js Route Handlers (Node)	Server actions for analysis, alert detail, quick fixes.
Database	PostgreSQL through Drizzle ORM	sales_micro_event, conception_alert, conception_recommendation, conception_security_block tables.
LLM (primary)	OpenRouter Chat Completions API	JSON-mode completions for the conception analysis pass.
LLM (fallback)	Google Generative Language API (Gemini 2.0 Flash)	Used when OpenRouter returns credit / quota errors.
Validation	Zod schemas	Hard schema for LLM output (severity, priority, ranges) before persisting.
Auth	Better Auth (server)	Session + privileged admin email gating for /seller-helper and admin APIs.
Telemetry contract	pa_* event whitelist (event-contract.ts)	Stable funnel + interaction event names from the storefront.
Pricing / catalog	product-content.ts, product-page-link.ts	Parses product description blocks, builds canonical product URLs.
Image rendering	next/image	Optimised thumbnails in the Vitrina cards (64×64) and modals.
Internationalisation	Intl.NumberFormat + Intl.RelativeTimeFormat	Dashboard uses en-US; security tab uses fr-FR (Algerian merchant audience).

Environment configuration

Variable	Required for	Notes
OPENROUTER_API_KEY	AI Recommendations (primary LLM)	Without it, only rule-engine alerts and Vitrina heuristics are produced.
CONCEPTION_OPENROUTER_MODEL	AI Recommendations (override)	Falls back to ASSISTANT_FREEFLOW_MODEL, then google/gemini-2.0-flash-001.
GOOGLE_API_KEY	AI Recommendations (Gemini fallback)	Used when OpenRouter fails with 402/insufficient credits.
CONCEPTION_GEMINI_MODEL	AI Recommendations (Gemini override)	Defaults to gemini-2.0-flash.
CONCEPTION_LLM_GEMINI_FALLBACK	Override fallback policy	Set to 'true' to always fall back, 'false' to never fall back.
DATABASE_URL	All Seller Helper data	PostgreSQL connection used by Drizzle in src/server/db.
BETTER_AUTH_URL / NEXT_PUBLIC_APP_URL	OpenRouter HTTP-Referer header	Identifies the calling app on OpenRouter.

Access control and routing policy

The route guards itself before rendering a single byte to the client:

1. `auth.api.getSession()` resolves the current Better Auth session. Missing session → redirect to `/signin`.
2. The user's row is loaded from `user` to read the `role` column.
3. Admin is `role === "admin" || isPrivilegedAdminEmail(email)`. Non-admins receive a stub page with links back to `/my-account` and `/` — *no telemetry payload* ever reaches their browser.
4. Only when admin is true does the page prefetch the overview, alerts, dismissed alerts and Vitrina recommendations in parallel via `Promise.all`.

Errors during the prefetch are captured as a string and rendered as a non-blocking banner; the dashboard still hydrates with whatever data was available.

Data foundation: events, alerts, security blocks, applied actions

All Seller Helper computations rely on five tables:

- **sales_micro_event** — the storefront micro-event log (pa_* events). Columns used: `session_key`, `event_name`, `payload_json`, `page_path`, `product_local_id`, `created_at`.
- **conception_alert** — persistent alerts emitted by the rule engine and the LLM. Each row has a unique `fingerprint`, severity, optional metadata JSON and an `affected_sessions_estimate`.
- **conception_recommendation** — long-form recommendations with priority, impactLabel, analysis, recommendation, confidence and an evidenceJson trail.
- **conception_security_block** — active session blocklist. The unique key is `session_key` with a nullable `lifted_at` field — an open block has `lifted_at IS NULL`.
- **seller_helper_applied_action** — audit log of operator actions taken inside the Seller Helper (Vitrina quick fixes, security blocks, alert resolutions, AI recommendations marked as applied). Indexed on `occurred_at`, `product_local_id` and `kind`. Powers the Timeline checkpoint overlay.

The canonical event vocabulary lives in `src/server/conception/event-contract.ts`:

```
export const STORE_EVENT = {
  productView: "pa_product_view",
  addToCart: "pa_add_to_cart",
  beginCheckout: "pa_begin_checkout",
  purchase: "pa_purchase",
  globalContext: "pa_global_context",
  pagePerformance: "pa_performance",
} as const;

export const PA_JS_ERROR = "pa_js_error" as const;
```

Anything outside this whitelist is intentionally invisible to the dashboard.

Dashboard (Overview) tab

The Overview tab is the landing experience. It composes four panels:

- **KPI strip (top, 4 cards)** — labels, values and a delta line marked positive/negative against the previous period. Drives the "is the store healthy?" judgement in under a second.
- **Traffic & Sales (24 h)** — a custom SVG `TrafficChart` driven by normalised hourly counts from `trafficHourlyNormalized()`. Counts are scaled to the local max so the visual is stable regardless of absolute volume.
- **Devices** — distribution slices parsed from the `device` field embedded in `payload_json` (mobile / tablet / desktop residual).
- **Top performing pages** — top page paths by view count, joined with their add-to-cart events to compute a per-page conversion rate.

An empty-data fallback (*No metrics available yet, No pages with recorded traffic*) is rendered whenever the query returns zero rows — the dashboard never breaks on a fresh deployment.

Timeline tab

The Timeline tab plots any combination of metrics over time. The X axis is always time; the Y axis is whatever the operator picks. Two Y axes are used simultaneously so volume (counts) and quality (percentages) can be compared on the same chart. Overlaid on top of the curves are *checkpoint markers* — small flag-shaped pins that mark the exact time an action was taken in any of the recommendation surfaces.

Controls

- **Range** — 24h (hourly buckets), 7d / 30d / 90d (daily buckets). Buckets are aligned to the current UTC hour or UTC day.
- **Scope** — Whole store, or a specific product. Per-product mode uses the same alias resolution as the heatmap (storefront product ID + DB hash aliases).
- **Metrics** — multi-select chips. The default set is views, add-to-carts, and purchases. Hovering each chip shows the metric's exact definition.

Metrics catalogue

ID	Label	Unit	Definition
views	Product views	count	pa_product_view events per bucket.
uniqueSessions	Unique sessions	count	Distinct session_key values per bucket.
addToCarts	Add to cart	count	pa_add_to_cart events per bucket.
purchases	Sales	count	pa_purchase events per bucket.
conversionRate	Conversion rate	percent	100 × purchases / views per bucket (0 if views=0).

Checkpoint markers (applied-action overlay)

Every time an operator takes an action against a recommendation, a checkpoint is written to `seller_helper_applied_action` and rendered as a pin at its exact occurrence time on the Timeline chart. Hovering a pin reveals the action kind, exact timestamp and (when applicable) the product. Clicking the pin opens a details modal carrying the full recommendation context that was used at the moment of application.

Kind	Trigger	Sourced from	Scope
Vitrina quick fix	Vitrina quick-fix application from Seller Helper > Vitrina tab	applyVitrinaQuickFixes()	Per-product
Recommendation applied	AI recommendation dismissed (marked as applied) on the Alerts/Recommendations panel	dismissConceptionRecommendationById()	Store-wide
Alert resolved	Conception alert marked as resolved	dismissConceptionAlertById() with disposition=resolved	Store-wide
Session blocked	Security quick-fix block_session	applySecurityQuickFixes()	Store-wide
Session unblocked	Security quick-fix unblock_session	applySecurityQuickFixes()	Store-wide

Server pipeline

1. `bucketsForRange` aligns the current time to UTC hour/day and emits N evenly-spaced bucket boundaries.
2. `loadEventCounts` runs one GROUP BY query on `sales_micro_event` with `date_trunc` and per-event filters (FILTER WHERE) producing one row per bucket and per metric counter.
3. For per-product scope, an `IN` filter on `product_local_id` uses the alias set returned by `getCatalogProductAliasIds()`.
4. Rate metrics (conversion rate) are computed at the bucket level — and at the window level for the KPI strip (sum of numerator / sum of denominator).
5. `listAppliedActionsInRange` reads the `seller_helper_applied_action` audit table in the same window. Per-product timelines include both product-bound and store-wide checkpoints so operators see all signals competing for attribution.

Below the chart, a three-card KPI strip surfaces, per active metric, the *total* (or window rate for percent metrics), the *average per bucket*, the *peak value* and the *peak bucket label*. The chart itself is a custom SVG line chart with hover tooltip and dual Y-axes — count axis on the left, percent axis on the right — so volume and rate metrics can coexist without distorting each other. Checkpoints sit above the curves with a small color-coded legend that summarizes how many of each kind occurred inside the visible window.

API surface:

```
GET /api/admin/seller-helper/timeline
?range=24h|7d|30d|90d
&scope=store|product
&productId=<storefront product id> # required when scope=product
&metrics=views,addToCarts,purchases # comma-separated

# Response envelope (subset)
{
  "timeline": {
    "range": "24h",
    "scope": "store",
    "granularity": "hour",
    "buckets": ["2026-05-12T13:00:00.000Z", ...],
    "series": [{ "metric": "views", "values": [...], ... }],
    "appliedActions": [
      {
        "id": "uuid",
        "kind": "vitrina_quick_fix" | "ai_recommendation" | "alert_resolved" | "security_block" | "security_unblock",
        "kindLabel": "Vitrina quick fix",
        "title": "Vitrina quick fix · Product name",
        "summary": "Promote color • Add availability note",
        "occurredAt": "2026-05-12T15:43:00.000Z",
        "productId": 123 | null,
        "productTitle": "Product name" | null,
        "sourceRefId": "<uuid or session key>",
        "details": { ... }
      }
    ]
  }
}
```

```
GET /api/admin/seller-helper/timeline/products
?windowDays=30 # ranks products by total views
```

User Behavior tab

The Behavior tab renders the qualitative side of telemetry:

- **Heatmap bands** — vertical bands shaded by intensity derived from `pa_scroll` depth distributions, attached to the most-viewed product page.
- **Product page heatmap** — an SVG composite from `ProductPageHeatmap.tsx` that overlays pointer activity zones on a synthetic product layout.
- **Journeys** — most frequent session journeys reconstructed from the funnel events.
- **Scroll depth metrics** — the proportion of sessions reaching 75% scroll.
- **Session replays** — non-video summaries: duration, device, status — opening the admin analytics for full event-by-event detail.

Cross-navigation is wired through `onNavigateSection` — clicking *View funnel* on a behaviour insight jumps to the Conversion Funnel tab without re-fetching.

Conversion Funnel tab

The funnel computes nested-subset counts over the last 7 days. The CTE is intentionally written so that each level is the intersection with the previous one — i.e. `n_cart` only counts sessions that *also* had a `pa_product_view`.

```
WITH pe AS (SELECT DISTINCT session_key FROM sales_micro_event WHERE event_name = 'pa_product_view' AND created_at >=
since),
    cart AS (SELECT DISTINCT session_key FROM sales_micro_event WHERE event_name = 'pa_add_to_cart' AND created_at >=
since),
    chk AS (SELECT DISTINCT session_key FROM sales_micro_event WHERE (event_name = 'pa_begin_checkout' OR page_path LIKE
'%checkout%') AND created_at >= since),
    fin AS (SELECT DISTINCT session_key FROM sales_micro_event WHERE event_name = 'pa_purchase' AND created_at >=
since)
SELECT
    (SELECT COUNT(*) FROM pe) AS
n_product,
    (SELECT COUNT(*) FROM pe JOIN cart USING (session_key)) AS
n_cart,
    (SELECT COUNT(*) FROM pe JOIN cart USING (session_key) JOIN chk USING (session_key)) AS
n_checkout_path,
    (SELECT COUNT(*) FROM pe JOIN cart USING (session_key) JOIN fin USING (session_key)) AS
n_final;
```

Friction items are then computed from the deltas between adjacent steps. The same numbers drive the recommendation engine — see the next chapter on the rule engine for the exact thresholds.

Step	Source event(s)	Position in funnel
Product view	pa_product_view	Entry
Add to cart	pa_add_to_cart	Intent
Begin checkout	pa_begin_checkout OR page_path LIKE '%checkout%'	Commit
Purchase	pa_purchase	Conversion

Vitrina Recommendation tab

Vitrina is the merchandising lens. It enumerates the catalogue, builds a per-product interaction snapshot over the last 7 days, computes deterministic tips, ranks the products, and renders them in two views: a featured Swiper carousel and a filtered grid with search / category / sort controls.

Pipeline

1. **Load catalogue rows** from `productsTable` joined with `categoryTable` ; cross-reference with the storefront product IDs from the catalogue helper.
2. **Aggregate signals** per storefront product over 7 days using a single GROUP BY query (views, hovers, clicks, add-to-carts, option selects, image views, specs interactions) plus a detail pass for view dwell time, scroll depth, click Y%, hover Y%, selected colours and review interactions.
3. **Build display snapshot** — title hints (colour, price, brand), `descriptionLength`, rating, stock.
4. **Build interaction snapshot** — window-normalised metrics, avg click Y%, avg hover Y%, top selected colours, `viewToCartRate`, `clickToCartRate`.
5. **Build tips** — see the heuristics table below.
6. **Build quick fixes** — at most four whitelisted IDs.
7. **Score and rank** by opportunity score (priority + traffic weight + friction weight).

Heuristic tip catalogue

Tip	Priority logic	Trigger	Action
Price tip	high / medium	views ≥ 6 AND ((viewToCartRate $< 10\%$) OR (priceZoneClicks ≥ 3 AND addToCarts low)) — price-sensitive case; or views ≥ 6 AND viewToCartRate $\geq 10\%$ with an active <code>jomlaPrice</code> — well-priced case.	Surface the selling price on the catalog thumbnail; if no Vitrina promo price is set, suggest enabling one via the <code>promo_price</code> quick fix.
Quantity & availability	high / medium	product.instock > 0 AND (views ≥ 10 OR hovers ≥ 12 OR clicks ≥ 8 OR baseline views ≥ 6)	Display in-stock copy on thumbnail and detail page; severity goes high when only 5 or fewer units remain while demand rises.
Quality concern	high / medium	reviewInteractions ≥ 3 OR (product.rating between 0 and 3.5 AND views ≥ 5)	Highlight rating proof and reassurance near the CTA; the priority promotes to high once five or more shoppers open the reviews section.

Quick fixes catalogue

ID	Label	Summary	Effect on DB
default_color	Default color	Move the most-selected colour to the first option so the PDP opens on the preferred shade.	Reorders the colours array in the product description (<code>parseProductContent</code> / <code>serializeProductContent</code>).
promo_price	Vitrina promo price	Set <code>jomlaPrice</code> to <code>price ÷ (1 + 0.20)</code> so the storefront shows a Vitrina promo with the original as strikethrough.	Writes a new <code>jomlaPrice</code> on <code>products.jomla_price</code> .

ID	Label	Summary	Effect on DB
availability_note	Availability message	Upsert an Availability entry — 'In stock — N unit(s) ready to ship.' — in the product description.	Writes the Availability additional-info key on products.description.
quality_highlight	Quality reassurance	Upsert a Quality entry showing the current rating (or a generic reassurance line if there's none yet).	Writes the Quality additional-info key on products.description.

The LLM pass for Vitrina is optional — its prompts are reproduced in the Prompt Library chapter — and it consumes the top three opportunity-ranked products only, to keep the JSON payload small and the response deterministic.

AI Recommendations tab

This tab is the human-readable surface of the LLM analysis. Each card is a `conception_recommendation` row with: priority, impact label, title, analysis, recommendation, confidence (0–100), and optional `revenueHint` / `implementationHint` / `roiHint`.

Operators can mark a recommendation as *Dismissed*; the UI sorts cards by importance and supports jumping to the section that would implement the change (Vitrine, Funnel, Behavior, Alerts) via `resolveImplementationSection()` — a keyword router that reads the title and recommendation text in both English and French.

The deterministic funnel-based recommendations always appear too. They are reproduced below for transparency:

Code	Priority	Trigger	Impact label	Confidence	Recommendation
REC_FUNNEL_PRODUCT_CART	high	nCart / nProduct < 0.35 (with nProduct > 0)	+4–8% conversion (est.)	82	Clarify all-in price, availability, and shipping above the fold; strengthen reviews and guarantees near the primary CTA.
REC_FUNNEL_CART_CHECKOUT	high	nCheckoutPath / nCart < 0.45 (with nCart > 0)	+5–10% conversion (est.)	88	Enable guest checkout, reduce form fields on mobile, show shipping costs early, and add a funnel progress bar.
REC_LCP_PERF	medium	lcpSlowSessions ≥ 3	+2–4% conversion (est.)	76	Compress visuals (WebP/AVIF), lazy-load below the fold, prioritise the hero, and limit third-party scripts on the product page.

Alerts tab

The Alerts tab consolidates active and dismissed incidents. Each row in `conception_alert` renders as a card carrying severity (critical / high / medium / low), title, description, optional detail, and an "Analyze in detail" action that calls `/api/admin/conception/alerts/detail`.

The detail endpoint runs `analyzeConceptionAlertById()` — a deterministic, per-type deep-dive function that produces:

- **Key indicators** — current vs reference values for the rate / volume / error count.
- **Deviations** — observed value vs baseline with a tone (critical/high/medium/low) computed from the delta.
- **Clues** — qualitative pointers (e.g. "Loss often concentrates between cart and checkout initiation").
- **Fix steps** — actionable checklist tailored to the alert family.

The dismissal flow accepts a disposition of `"resolved"` or `"ignored"`, persisted on the alert row; the right column lists the most recent 12 dismissed alerts so operators can compare incident lifetimes.

Security tab

The Security tab is built from a dedicated brief computed by `buildConceptionSecurityBrief()` :

- **4 KPI cards** — Bots détectés, Sessions bloquées, Tentatives de fraude, Score de sécurité.
- **Score formula card** — the formula is shown verbatim to operators (see below).
- **Threat activity (24 h)** — hourly series of events emitted by risky sessions; rendered through `ThreatActivityChart` .
- **Threat types (7 d)** — distribution of Bot scraping / Click fraud / Fausses commandes / Erreurs JS.
- **Active incidents** — top 8 risky sessions, masked identity, time-ago in French.
- **Blocked sessions table** — currently active blocks with reason and time-since-block.
- **Engine notes** — French-language narrative footnotes, always including the privacy reminder.

The visible score formula:

```
Score = 100 - min(30, bots/sessions×35) - min(18, blocages/sessions×20) - min(24, fraudes×4) - min(16, pic horaire/80×16),  
borné entre 0 et 100.
```

Per-session risk patterns

Pattern	Trigger	Score contribution
Bot scraping (high velocity)	events ≥ 50 AND 0 < duration_s ≤ 360	+40
Bot scraping (catalogue scrape)	product_views ≥ 25 AND 0 < duration_s ≤ 600	+25
Click fraud	clicks ≥ 35 AND 0 < duration_s ≤ 300	+20
Fake checkout	checkouts > 0 AND no pa_product_view in the same session	+15
JS error storm	js_errors ≥ 3	+10
Clickfarm pattern (low engagement)	hovers ≤ 1 AND clicks ≥ 10	+8

Security quick fixes

ID	Label	Summary	Effect on DB
block_session	Bloquer la session	Adds the session_key to conception_security_block with a manual reason.	Upserts a row; on conflict, the existing row is reactivated (blockedAt reset, liftedAt cleared).
unblock_session	Débloquer la session	Lifts an active block by setting conception_security_block.lifted_at = now().	Only the current open block (lifted_at IS NULL) is updated.

Both quick fixes are confirmed in a modal before being committed. Blocks are upserted (so re-blocking a previously lifted session reactivates the row); unblocks only update rows that are currently open.

Rule engine — full catalogue

The deterministic rule engine lives in `src/server/conception/analyze.ts` and runs on every *Run analysis* click. Each rule emits an alert with a unique fingerprint so re-runs don't duplicate.

CONVERSION_DROP

CRITICAL

Window: 7 days vs previous 7 days

Trigger: $\text{rateNow} < 0.8 \times \text{rateOld}$ (and $\text{rateOld} > 0.001$)

Fingerprint: Daily — one alert per UTC day.

Conversion rate has dropped more than 20% versus the rolling reference. Detail captures both rates so operators can compare instantly.

TRAFFIC_SPIKE

HIGH

Window: Last 15 minutes vs 90-minute baseline

Trigger: $\text{events15m} > \text{baseline15} \times 4$

Fingerprint: Hourly — one alert per UTC hour.

Event density at least 4× the 90-minute average over a 15-minute window. Hourly fingerprint avoids spam during sustained spikes.

CART_ABANDON_MASS

MEDIUM

Window: Rolling 2 hours

Trigger: $\text{cart2h} \geq 8$ AND $\text{cartAbandon2h} \geq 0.8$

Fingerprint: Hourly.

At least 8 sessions added to cart in 2 hours and 80%+ never reached `pa_purchase`. $\text{affectedSessionsEstimate} = \text{cart2h} - \text{final2h}$.

JS_ERROR_BURST

HIGH

Window: Rolling 2 hours

Trigger: $\text{jsErrorSessions} / \text{sessionsCheckout2h} \geq 0.05$

Fingerprint: Hourly.

More than 5% of checkout sessions raise `pa_js_error` events — usually a regression on the checkout page.

PERF_SLOW

LOW

Window: Rolling 2 hours

Trigger: $\text{slowNavSessions} + \text{lcpSlowSessions} \geq 5$

Fingerprint: Hourly.

Five or more sessions show slow navigation or LCP > 4 s, often image weight on the product page.

Deterministic funnel recommendations

The same analysis run also writes three deterministic recommendations when the funnel ratios fall under thresholds:

Code	Priority	Trigger	Impact label	Confidence	Recommendation
REC_FUNNEL_PRODUCT_CART	high	$\text{nCart} / \text{nProduct} < 0.35$ (with $\text{nProduct} > 0$)	+4–8% conversion (est.)	82	Clarify all-in price, availability, and shipping above the fold; strengthen reviews and guarantees near the primary CTA.
REC_FUNNEL_CART_CHECKOUT	high	$\text{nCheckoutPath} / \text{nCart} < 0.45$ (with $\text{nCart} > 0$)	+5–10% conversion (est.)	88	Enable guest checkout, reduce form fields on mobile, show shipping costs early, and add a funnel progress bar.
REC_LCP_PERF	medium	$\text{lcpSlowSessions} \geq 3$	+2–4% conversion (est.)	76	Compress visuals (WebP/AVIF), lazy-load below the fold, prioritise the hero, and limit

Code	Priority	Trigger	Impact label	Confidence	Recommendation
					third-party scripts on the product page.

LLM strategy and provider fallback

The LLM pass is orchestrated by `runConceptionLlmAnalysis()`. Its goal is to produce a French operator summary plus structured alerts and recommendations grounded in the current telemetry snapshot, never inventing metrics that aren't in the payload.

Provider chain

1. **OpenRouter** (primary) — model picked from `CONCEPTION_OPENROUTER_MODEL` → `ASSISTANT_FREEFLOW_MODEL` → `google/gemini-2.0-flash-001`. JSON mode is enforced via `response_format: { type: "json_object" }`. Up to 3 attempts with 1.2 s × attempt back-off on 429/5xx.
2. **Gemini 2.0 Flash** (fallback) — used when OpenRouter fails with a 402 (insufficient credits) or when `CONCEPTION_LLM_GEMINI_FALLBACK=true`. Calls `generativelanguage.googleapis.com` with `responseMimeType: "application/json"`.

Validation pipeline

1. `extractJsonObject()` tolerates wrapping prose / markdown fences.
2. `normalizeAnalysisPayload()` accepts both English and French keys (`alerts` / `alertes` , `recommendations` / `recommandations` , `analysis` / `analyse`) and coerces severity / priority through fuzzy matching (`crit` , `haute` , `moy`).
3. `llmAnalysisSchema` (Zod) enforces `severity ∈ {critical, high, medium, low}` , `confidence ∈ [0, 100]` , max array sizes (6 alerts, 8 recommendations), and bounded string lengths.
4. `mapLlmOutput()` appends day-fingerprints (`LLM-ALERT-...-2026-05-12`) and serialises `metadata_json` / `evidence_json` for traceability.

Determinism levers

- Temperature is fixed at `0.2`.
- `max_tokens` is capped at 2 200 for OpenRouter.
- The LLM only ever sees: latest overview, top 6 KPIs, funnel + summary + friction items, top 6 pages, devices, security brief and the analyze signals.

Prompt library (verbatim)

This chapter quotes the prompts from source so reviewers can audit the exact instructions sent to the model. The strings are read from the repository at build time.

SRC/SERVER/CONCEPTION/LLM-ANALYSIS.TS Conception analysis — system prompt

You are a senior ecommerce analytics consultant for Vitrina Store.
 Analyze only the telemetry JSON provided by the user.
 Return one JSON object only with exactly these top-level keys: summary, alerts, recommendations.
 summary must be a plain French string, never an object.
 alerts must be an array. recommendations must be an array.
 severity must be exactly one of: critical, high, medium, low.
 priority must be exactly one of: critical, high, medium, low.
 confidence must be an integer from 0 to 100.
 Write summary, titles, descriptions, analysis, and recommendation fields in French.
 Do not invent metrics that are absent from the payload.
 Prefer actionable merchandising, conversion, checkout, performance, and security insights.
 Each alert must include alertType (short snake_case code), severity, title, description, optional detail, optional affectedSessionsEstimate.
 Each recommendation must include priority, impactLabel, title, analysis, recommendation, confidence, and optional revenueHint, implementationHint, roiHint.
 If event data is sparse, produce cautious recommendations and lower confidence instead of fabricating incidents.

SRC/SERVER/CONCEPTION/LLM-ANALYSIS.TS Conception analysis — user prompt template

Analyze this Seller Helper telemetry snapshot and return JSON only.

```

${JSON.stringify(context)}

```

SRC/SERVER/SELLER-HELPER/VITRINA-RECOMMENDATION-PROMPT.TS Vitrina merchandising — system prompt

You are a merchandising analyst for Vitrina Store.
 Use only the JSON payload provided by the user.
 Return one JSON object with exactly one key: recommendations.
 recommendations must be an array with at most the maxRecommendations value from the payload.
 Each recommendation must include:
 - productId (string, must match an input productId)
 - primaryRecommendation (one concise merchandising action in French)
 - tips (array of 1 to 3 objects with label, action, priority)
 - isTopRecommendation (boolean; exactly one item in the array must be true)
 priority must be exactly one of: high, medium, low.
 Ground every recommendation in the supplied display and interaction fields.
 Prioritize changes to title, main image, price visibility, default color, promo price, and catalog thumbnail messaging.
 If interaction data is sparse, lower confidence and avoid inventing metrics.
 Write primaryRecommendation, label, and action in French.

SRC/SERVER/SELLER-HELPER/VITRINA-RECOMMENDATION-PROMPT.TS

Vitrina merchandising — user prompt template

Generate storefront merchandising recommendations from this Seller Helper snapshot.
 Select only products that need a visible merchandising change.

Return JSON only.

```
${JSON.stringify(payload)}
```

Both prompts insist on French operator copy, JSON-only output, and grounding in the supplied payload. Combined with the Zod schema, this gives the dashboard predictable, audit-ready recommendations.

Vitrina heuristics, scoring and ranking

Vitrina ranking is a three-component score in `opportunityScore()` :

```
opportunityScore = priorityWeight + trafficWeight + frictionWeight

priorityWeight = Σ (4 - IMPORTANCE_RANKS[tip.priority])    // high=3, medium=2, low=1
trafficWeight  = min(12, log2(views + 2) × 2)
frictionWeight = (viewToCartRate != null && viewToCartRate < 0.1)
                  ? (0.1 - viewToCartRate) × 40
                  : 0
```

Ties are broken by the strongest tip priority (lowest IMPORTANCE_RANK). The list is then filtered:

- Default view — any product with at least one tip.
- `actionableOnly` — products with at least one high or medium tip.

The featured Swiper at the top of the tab shows the top three opportunity-ranked products; the grid below allows search by title/category/brand/slug, a category filter, and six sort modes (opportunity / interaction / merchandising priority — high or low).

Per-product interaction signals

Signal	Source event	Used in
views	pa_product_view	opportunity score, price tip, quality tip
hovers	pa_pointer_hover (≠ heatmap_embed)	demand classification
clicks	pa_pointer_click (≠ heatmap_embed)	demand + click Y% zones
addToCarts	pa_add_to_cart	viewToCartRate, clickToCartRate
optionSelects	pa_select_option	default_color quick fix candidate
imageViews	pa_image_view_time	interaction score
specsInteractions	pa_specs_interaction	interaction score
specsDwellMs	pa_specs_view_time (visible_ms)	engagement signal
scrollDepth75Plus	pa_scroll (depth_pct ≥ 75)	engagement signal
clickYTotal / clickYCount	pa_pointer_click (y_pct)	avg click Y, priceZoneClicks
hoverYTotal / hoverYCount	pa_pointer_hover (y_pct)	avg hover Y
reviewInteractions	pa_review*	quality concern tip
priceZoneClicks	pa_pointer_click with y_pct ≤ 35	price tip (price-sensitive case)

Quick fixes (Vitrina and Security)

Quick fixes are the only mutations the dashboard can apply. They share a strict contract:

1. The client sends a structured payload (a list of `{ id, label, summary, context }`).
2. The server validates each `id` against a whitelist of `VitrinaQuickFixId` or `ConceptionSecurityQuickFixId`.
3. For Vitrina, the server re-reads the recommendation, so the actual fixes applied must match what was offered — no smuggling of new fix IDs.
4. Mutations are scoped to a tiny set of columns; nothing is destructive.

Vitrina catalogue mutations

ID	Label	Summary	Effect on DB
default_color	Default color	Move the most-selected colour to the first option so the PDP opens on the preferred shade.	Reorders the colours array in the product description (<code>parseProductContent / serializeProductContent</code>).
promo_price	Vitrina promo price	Set <code>jomlaPrice</code> to <code>price ÷ (1 + 0.20)</code> so the storefront shows a Vitrina promo with the original as strikethrough.	Writes a new <code>jomlaPrice</code> on <code>products.jomla_price</code> .
availability_note	Availability message	Upsert an Availability entry — 'In stock — N unit(s) ready to ship.' — in the product description.	Writes the Availability additional-info key on <code>products.description</code> .
quality_highlight	Quality reassurance	Upsert a Quality entry showing the current rating (or a generic reassurance line if there's none yet).	Writes the Quality additional-info key on <code>products.description</code> .

Internal constants:

- `VITRINA_STANDARD_MARKUP = 0.20` — used by the *promo_price* fix: `jomlaPrice = round(price / 1.20)` with a minimum of 1.
- `AVAILABILITY_KEY = "Availability"`, `QUALITY_KEY = "Quality"` — keys upserted in the product description's additional-info block.
- Maximum 4 quick fixes per request.

Security mutations

ID	Label	Summary	Effect on DB
block_session	Bloquer la session	Adds the <code>session_key</code> to <code>conception_security_block</code> with a manual reason.	Upserts a row; on conflict, the existing row is reactivated (<code>blockedAt</code> reset, <code>liftedAt</code> cleared).
unblock_session	Débloquer la session	Lifts an active block by setting <code>conception_security_block.lifted_at = now()</code> .	Only the current open block (<code>lifted_at</code> IS NULL) is updated.

Security mutations also surface a confirmation modal and persist the reason in `conception_security_block.reason` (default fallback: *"Blocage manuel"*).

Refresh model and side effects

The dashboard maintains three explicit state-flow primitives in `useConceptionAdminData` :

- **refresh()** — re-fetches the overview + alert lists + Vitrina list without touching the LLM. Cheap and idempotent.
- **runAnalyze()** — calls the analyze endpoint, which runs the rule engine, the LLM analysis (with provider fallback), and re-lists Vitrina. The hero button shows *Analyzing...* while busy. Returns the count of inserted alerts/recommendations and the LLM summary (if any).
- **dismissAlert / dismissRecommendation** — optimistic UI dismissal with a server confirmation.

The hero status badge switches between *Live data · auto-refresh* (when `overview.hasEventData` is true) and *Waiting for data · auto-refresh*. The page is marked `export const dynamic = "force-dynamic"` so it never serves stale SSR cache.

Privacy, integrity and operational policies

The following policies are enforced in code, not just in documentation. Each is observable in the source files listed below.

Privileged access only

/seller-helper resolves the current Better Auth session, joins user.role, and unions with the privileged admin email list. Anything that is not admin sees a stub page with links back to /my-account and / — no metrics are shipped to the client.

Server-rendered data, hydrated once

The page server-fetches the conception overview, active alerts, dismissed alerts and Vitrina recommendations in parallel (Promise.all). The client receives a typed ConceptionAdminInitialData payload and uses it as the initial state of useConceptionAdminData — no waterfall on first paint.

Schema-validated LLM output

Every LLM response goes through extractJsonObject, a permissive normaliser (English/French keys), then a Zod schema with hard enums (severity, priority) and clamped numeric ranges. If validation fails, the response is rejected — never written to the DB.

Idempotent, deduplicated writes

Alerts and recommendations use stable fingerprints (CONVERSION_DROP-2026-05-12, REC_FUNNEL_CART_CHECKOUT-2026-05-12). Inserts are onConflictDoNothing against the fingerprint, so re-running the analysis job is safe.

No raw PII in micro-events

session_key is an opaque identifier. The security brief explicitly notes: 'Les identités client sont dérivées des clés de session — aucune adresse IP n'est stockée dans les micro-événements.' Display identities are masked (sliced + ellipsis).

Bounded prompts and JSON-only contract

Both the conception system prompt and the Vitrina prompt force a single top-level JSON object with explicit keys, capped arrays (≤ 6 alerts, ≤ 8 recommendations, ≤ 3 Vitrina products), bounded string lengths and confidence $\in [0, 100]$. Temperature is fixed at 0.2.

Provider fallback with retry budget

OpenRouter is tried up to 3 times with exponential backoff on 429/5xx. A 402 (insufficient credits) automatically routes the same prompt to Gemini, unless CONCEPTION_LLM_GEMINI_FALLBACK overrides this default.

Localised seller voice

Dashboard copy is in English (en-US), but recommendation prose written by the LLM and the entire Security tab are forced to French — the Algerian merchant audience reads operational text in French and DA prices.

Quick fixes are scoped and reversible

Vitrina quick fixes can only mutate four whitelisted fields (jomla_price, description.colors, Availability, Quality). Security quick fixes can only block or unblock a session — never delete events.

Operationally, the recommended runbook is:

1. Run a daily *Run analysis* per store, or trigger it from a scheduled job hitting the same endpoint.

2. Treat fingerprints as authoritative deduplication keys; never alter them in migrations.
3. If OpenRouter credit is depleted, the dashboard remains functional through Gemini fallback — monitor the `llmModel` field returned from `analyze` to know which provider answered.
4. When extending the prompts, update the Zod schema and the normaliser at the same time; both must accept the new shape.

Appendices

Reference tables to keep this document auditable against the repository.

Appendix A — Source file index

File	Role
src/app/(site)/(pages)/seller-helper/page.tsx	Server page — auth, admin gating, prefetch.
src/components/SellerHelper/SellerHelperDashboard.tsx	Client dashboard shell, hero, nav and tab switcher.
src/components/SellerHelper/nav.ts	Nav item enumeration and tab metadata (label + description).
src/components/SellerHelper/layout.ts	Tailwind class primitives shared across all tabs.
src/components/SellerHelper/sections.tsx	User Behavior, Alerts and AI Recommendations tab content.
src/components/SellerHelper/timeline-tab.tsx	Timeline tab — metric/scope/range selectors + big chart + KPI strip.
src/server/seller-helper/timeline-series.ts	Bucketed timeline aggregator (whole store or per-product).
src/types/seller-helper-timeline.ts	Timeline metric, range, scope, DTO definitions and applied-action kind catalogue.
src/app/api/admin/seller-helper/timeline/route.ts	GET endpoint returning the bucketed timeline DTO.
src/app/api/admin/seller-helper/timeline/products/route.ts	GET endpoint listing products available for per-product timelines.
src/server/seller-helper/applied-actions.ts	Audit logger + range reader for Timeline checkpoint markers.
src/components/SellerHelper/AppliedActionDetailsModal.tsx	Modal that renders full context for a checkpoint when clicked on the chart.
scripts/ensure-seller-helper-applied-action.mjs	Idempotent DDL script that ensures seller_helper_applied_action exists.
src/components/SellerHelper/vitrina-recommendations.tsx	Vitrina tab — search/filter/sort + featured Swiper + grid.
src/components/SellerHelper/security-tab.tsx	Security tab — KPIs, threats, incidents and blocked sessions.
src/components/SellerHelper/charts.tsx	Custom SVG charts (TrafficChart, ThreatActivityChart, ProgressBar).
src/components/SellerHelper/ProductPageHeatmap.tsx	Product page heatmap visualisation.
src/components/SellerHelper/VitrinaQuickEditModal.tsx	Inline product editor (title, price, description, colours...).
src/components/SellerHelper/VitrinaQuickFixConfirmModal.tsx	Confirmation modal for Vitrina quick fixes.
src/components/SellerHelper/SecurityQuickFixConfirmModal.tsx	Confirmation modal for security block / unblock.
src/hooks/useConceptionAdminData.ts	Hook that hydrates overview/alerts/recommendations and exposes actions.
src/server/conception/metrics.ts	Funnel, KPIs, devices, top pages and overview DTO builder.
src/server/conception/analyze.ts	Rule-engine + LLM orchestrator (runConceptionAnalysisJob).
src/server/conception/llm-analysis.ts	OpenRouter + Gemini analysis pipeline with Zod validation.
src/server/conception/alert-detail-analysis.ts	Per-alert deterministic deep-dive used by the Alerts modal.
src/server/conception/alert-rules.ts	Catalogue of active rules surfaced on the Alerts tab.
src/server/conception/security-intel.ts	Security brief — KPIs, threats 24 h chart, incidents, blocks.
src/server/conception/apply-security-quick-fixes.ts	Server logic for block_session / unblock_session quick fixes.

File	Role
src/server/seller-helper/product-marketing-recommendations.ts	Heuristic Vitrina recommendations and signal aggregation.
src/server/seller-helper/vitrina-recommendation-prompt.ts	Prompt builder for the Vitrina LLM merchandising pass.
src/server/seller-helper/apply-vitrina-quick-fixes.ts	Server logic for the 4 Vitrina quick fixes.
src/server/conception/event-contract.ts	STORE_EVENT enumeration and PA_JS_ERROR constant.
src/server/lib/openrouter-client.ts	OpenRouter HTTP client used by the LLM analysis pipeline.

Appendix B — Environment variables

Variable	Required for	Notes
OPENROUTER_API_KEY	AI Recommendations (primary LLM)	Without it, only rule-engine alerts and Vitrina heuristics are produced.
CONCEPTION_OPENROUTER_MODEL	AI Recommendations (override)	Falls back to ASSISTANT_FREEFLOW_MODEL, then google/gemini-2.0-flash-001.
GOOGLE_API_KEY	AI Recommendations (Gemini fallback)	Used when OpenRouter fails with 402/insufficient credits.
CONCEPTION_GEMINI_MODEL	AI Recommendations (Gemini override)	Defaults to gemini-2.0-flash.
CONCEPTION_LLM_GEMINI_FALLBACK	Override fallback policy	Set to 'true' to always fall back, 'false' to never fall back.
DATABASE_URL	All Seller Helper data	PostgreSQL connection used by Drizzle in src/server/db.
BETTER_AUTH_URL / NEXT_PUBLIC_APP_URL	OpenRouter HTTP-Referer header	Identifies the calling app on OpenRouter.

Appendix C — Glossary

Term	Definition
Seller Helper	Admin-only operations console for store owners. Combines analytics, AI recommendations, alerts, and security tools behind one URL: /seller-helper.
Conception engine	Internal name for the rule + LLM pipeline that emits alerts and recommendations from pa_* micro-events.
Vitrina recommendations	Per-product merchandising tips computed from interaction signals — drives the Vitrina tab.
pa_* events	Storefront micro-events (pa_product_view, pa_pointer_click, pa_add_to_cart...). The canonical telemetry contract.
session_key	Opaque per-session identifier. No IP or raw user data is persisted in micro-events.
Quick fix	A small, reversible mutation triggered from the dashboard — Vitrina or Security.
Fingerprint	Deterministic string used to deduplicate alerts and recommendations across re-runs.
Opportunity score	Composite Vitrina score combining priority weight + log(views) traffic + viewToCart friction.
Risk score	Per-session security score in [0, 100] — adds heuristics for bot scraping, click fraud, fake checkout, JS errors.
Importance ranking	Shared scale (critical, high, medium, low) used to sort friction items, alerts and recommendations.

Colophon. Typography pairs DM Sans (headings, UI) with Source Serif 4 (body). Diagrams use plain figures for portable PDF rendering. Prompts are read from `src/server/conception/llm-analysis.ts` and `src/server/seller-helper/vitrina-recommendation-prompt.ts` at build time. To regenerate: `node scripts/build-seller-helper-report.mjs`, then `node scripts/build-seller-helper-report.mjs --pdf` with Playwright installed.

